

Typing Speed & Accuracy Test

A Python-powered GUI application designed to help users measure and improve their typing skills through real-time testing and instant feedback.

Presented by Team Accuracy



Meet Team Accuracy

Kartik

S25CSEU0360

Team Lead

Aditya Arora

S25CSEU0351

Idea and Concept



Abhinav Akash

S25CSEU0360 **Tech Member**



Arnav Sharma

S25CSEU0343 **PPT Maker**

Our collaborative team brought together diverse skills in Python programming, and quality assurance to create an intuitive typing test application.

Project Overview

What We Built

Our typing speed test is an interactive Python program that evaluates both speed and accuracy in real-time. The application provides instant feedback on performance metrics, helping users understand their typing capabilities.

Key Features

- Real-time typing speed measurement in words per minute (WPM)
- Precision accuracy calculation based on character matching
- User-friendly Tkinter GUI with intuitive controls
- Instant results display with detailed performance metrics



Technologies Used

Python

Core programming language providing the logic, algorithms, and computational foundation for the entire application.

Tkinter

Built-in Python GUI library used to create the interactive user interface with buttons, text fields, and labels.

Time Module

Essential for tracking typing duration with precision timing capabilities, measuring performance down to milliseconds.

Random Module

Generates varied test sentences from a predefined pool, ensuring each typing test offers a unique challenge.

How the Program Works

01

Launch & Start

User opens the application and clicks the 'Start Test' button to initiate a new typing challenge.

03

Auto Timer

The timer automatically begins counting the moment the first character is typed, ensuring accurate time tracking.

05

Calculate Results

Clicking 'Check Result' instantly analyzes the input, calculating WPM based on correct characters and total time elapsed.

02

Display Sentence

A random sentence appears on screen, selected from a diverse collection of practice texts.

04

User Input

User carefully types the displayed sentence in the input field, focusing on both speed and accuracy.

06

Display Metrics

Performance statistics appear immediately, showing time taken, typing speed, and accuracy percentage.

Program Code Structure

Code Implementation

The complete source code demonstrates clean Python programming practices with modular functions and clear logic flow. Our implementation includes:

```
import tkinter as tk
import time
import random

sentences = [
    "Hi, welcome to this program.",
    "Python is a beginner-friendly programming language.",
    "This program will help you check your typing accuracy.",
    "Consistency is the key to success.",
    "Test your typing speed.",
    "Developed by Team Accuracy."
]

start_time = 0
test_sentence = ""

def start_test():
    global start_time, test_sentence
    test_sentence = random.choice(sentences)
    sentence_label.config(text=test_sentence)
    typing_entry.delete(0, tk.END)
    typing_entry.config(state=tk.NORMAL)
    typing_entry.focus()
    start_time = time.time()
    result_label.config(text="")

def check_result():
    global start_time, test_sentence
    end_time = time.time()
    typed = typing_entry.get()
    total_time = round(end_time - start_time, 2)

    original_words = test_sentence.split()

    typed_words = typed.split()

    correct_words = sum(1 for i in range(min(len(original_words), len(typed_words))) if original_words[i] == typed_words[i])

    accuracy = (correct_words / len(original_words)) * 100
    wpm = (len(typed_words) / total_time) * 60 if total_time > 0 else 0

    result_label.config(text=f"Time Taken: {total_time} seconds\nTyping Speed: {wpm:.2f} WPM\nAccuracy: {accuracy:.2f}%")

root = tk.Tk()
root.title("Speed & Accuracy Test")
root.geometry("700x300")
root.configure(bg="black")

title_label = tk.Label(root, text="Typing Speed & Accuracy Test", font=("Arial", 18))
title_label.pack(pady=10)

sentence_label = tk.Label(root, text="Click 'Start Test' to begin", font=("Arial", 14), wraplength=650)
sentence_label.pack(pady=10)

typing_entry = tk.Entry(root, font=("Arial", 14), width=80, state=tk.DISABLED)
typing_entry.pack(pady=10)

start_button = tk.Button(root, text="Start Test", font=("Arial", 14), command=start_test)
start_button.pack(pady=5)

check_button = tk.Button(root, text="Check Result", font=("Arial", 14), command=check_result)
check_button.pack(pady=5)

result_label = tk.Label(root, text="", font=("Arial", 14))
result_label.pack(pady=10)

root.mainloop()
```

Output Example

12.4s

Time Taken

Total duration from first keystroke to test completion

35

Typing Speed

Words per minute (WPM) calculated from accurate characters

92%

Accuracy

Percentage of correctly typed characters vs. total characters

Results are displayed instantly after clicking 'Check Result', providing comprehensive feedback on typing performance. The accuracy metric helps users identify areas for improvement, while WPM offers a standardized measure of typing proficiency.

Conclusion & Future Enhancements

Project Summary

We successfully developed a functional and user-friendly typing test application using Python and Tkinter. The program provides accurate, real-time feedback on typing performance, making it an effective tool for skill assessment and improvement.

Future Possibilities

- Expanded sentence database with varying difficulty levels
- Progress tracking and historical performance graphs
- Customizable test durations and text categories
- Multiplayer mode for competitive typing challenges
- Export functionality for saving and sharing results



References



YouTube

Online tutorials and programming guides from various channels proved invaluable for understanding complex concepts and practical implementations.



College Notes

Academic materials and coursework from computer science studies formed the foundational knowledge for this project.



Think Python (2nd Edition)

This comprehensive guide to Python programming was a primary resource for language syntax, data structures, and best practices.

Thank You!

Thank you for your attention and time.

Team Accuracy